

Operating Systems

Assignment # 4

Benchmarking Sequential, Pthreads, OpenMP, and MPI Programs for Matrix Multiplication & Sorting

By: Namra Basharat
Roll No.: 476203
Course Instructor: Tanveer Ahmad
Date: 30 November 2025
Class: BS Bioinformatics UG-1

School of Interdisciplinary Engineering & Sciences
National University of Sciences and Technology

1 Introduction

Modern processors include multiple cores, hardware threads, and sometimes distributed systems. Software that runs in a strictly sequential manner uses only a fraction of available computational resources. When input sizes increase, the time required to complete a single-threaded program grows proportionally with its algorithmic complexity, and parallel approaches become attractive.

This report explores four execution models applied to the same computational workloads: basic sequential execution, POSIX threads (Pthreads), OpenMP, and MPI. Rather than focusing on compiler-level optimizations or numerical kernels, the work emphasizes experimentation. Each model was implemented in C, compiled with appropriate flags, and evaluated on identical input sizes. A set of scripts automates data generation, compilation, and benchmarking to reduce user bias.

The goal of the experiments is to observe how performance changes with input size and to understand the trade-offs involved in creating threads, sharing memory, and exchanging data between processes. The results reflect practical tendencies: parallelism improves runtime only when the amount of useful work is sufficient to offset the cost of managing concurrency.

The complete project source code, including all scripts, C implementations, and data generators, is publicly available on GitHub:

`https://github.com/Namra-5/parallel_sorting_and_matrix_benchmark.git`

2 Key Concepts

Parallel computing enables multiple computational units to cooperate on a task, often reducing execution time and allowing larger problem sizes than single-threaded execution can support. Each parallel model offers distinct capabilities and limitations.

Sequential Execution

Sequential execution processes instructions in a single thread without concurrency. It offers deterministic behavior and minimal scheduling overhead. Performance is fully constrained by the processor's single-core throughput and cache behavior. As input size grows, runtime increases predictably in proportion to algorithmic complexity. It establishes the baseline used to measure parallel speedups and overhead.

OpenMP

OpenMP is a pragma-based shared-memory parallel programming model. Parallel sections are expressed using compiler directives such as `#pragma omp parallel` and `#pragma omp for`. Developers annotate loops or regions, and the compiler generates multithreaded code.

This reduces implementation complexity and allows incremental parallelization. Performance depends on work decomposition, cache locality, and available cores. Over-scheduling can introduce thread management overhead, while poorly distributed workloads may cause idle cores and reduced efficiency. It assumes uniform memory access (SMP). Performance collapses when threads saturate memory bandwidth.

POSIX Threads (Pthreads)

Pthreads provide low-level control over thread creation, synchronization, and lifecycle. This enables fine-grained workload partitioning, custom schedulers, and more predictable memory usage. However, the developer must explicitly manage synchronization, race avoidance, and data ownership. Well-designed Pthreads implementations can outperform high-level frameworks, but they require significantly more effort and careful engineering. Its performance is sensitive to granularity: too small partitions then scheduling dominates whereas too large portions result in poor load balancing.

MPI

The Message Passing Interface enables distributed-memory parallelism by coordinating independent processes that communicate via message passing. It avoids shared-memory contention and can scale across nodes, making it suitable for very large problems. However, communicating intermediate results introduces latency, and poorly structured communication patterns can dominate runtime. When computation is large relative to communication, MPI can deliver strong scalability. MPI scales along node count (horizontal scaling), unlike threads which scale vertically on a single machine.

3 Matrix Multiplication

Given matrices $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$, their product $C = A \cdot B$ is defined element-wise as:

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}.$$

The classical algorithm computes each element of C independently, each involving n multiplications and additions, yielding $O(mnp)$ complexity. For square matrices of size $n \times n$, this becomes $O(n^3)$. This cubic growth makes matrix multiplication a suitable benchmark for evaluating execution models.
The comparison aims to demonstrate differences in scalability, thread-level parallelism, communication overhead, and performance behavior across matrix sizes ranging from very small matrices (5×4) to large dense matrices (5000×5000).

Dense matrix multiplication is compute-bound. Each element of C reuses an entire row of A and a column of B. Parallel models benefit because arithmetic dominates memory traffic;

communication overhead becomes negligible at larger sizes.

3.1 Experimental Setup

The matrix-multiplication performance study was conducted in a Linux environment using C and four execution models: Sequential, Pthreads, OpenMP, and MPI. The workflow was structured to ensure reproducibility, controlled benchmarking, and isolation of performance characteristics across models.

The repository was cloned and the compilation performed inside the `src/` directory using model-specific flags.

1. **Clone repository:**

```
> git clone https://github.com/mperlet/matrix_multiplication.git
> cd matrix_multiplication
```

i - Matrix Multiplication Repository:
A matrix multiplication environment obtained follows a modular structure that separates data generation, computation source files, and benchmarking scripts. The relevant files are organized as follows:

- **data/** : Contains generated matrix input files
- **Makefile** : Defines compilation targets for all versions `bin/seq`, `bin/omp`, `bin/thread2`, `bin/mpi`
- **Test-Script.sh** : Automation script for input generation, compilation, execution, and benchmarking
- **random_float_matrix.py** : Python matrix generator producing tab-separated floating-point values.
- **src/** : Contains matrix computation source C files:
  - `matrix.c` : Matrix parsing, allocation, printing
  - `sequential.c` : sequential implementation
  - `thread2.c` : Pthreads implementation
  - `omp.c` : OpenMP implementation
  - `mpi.c` : MPI implementation

** Only random_float_matrix.py and Test-Script.sh were modified, while the rest of the files were left unmodified to preserve reference implementations.*



2. Compile all four versions using gcc/mpicc or `make` as in the Makefile:

```
> gcc sequential.c matrix.c -o sequential.out
> gcc thread2.c matrix.c -o thread2.out -lpthread
> gcc omp.c matrix.c -o omp.out -fopenmp
> mpicc mpi.c matrix.c -o mpi.out
```

The resulting executables were stored in `bin/`.

3.2 Python matrix generator

The Python file is used to generate random matrices. The original repository used Python 2 syntax, thus the shebang, Exception and printing syntax were modified to work with Python 3. The generator performs three functions:

- Accepts matrix dimensions from the command line.
- Produces floating-point entries.
- Prints the matrix using plain text formatting without additional characters.

Modified Script : random_float_matrix.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
# this small helper script generates test matrices of the given
# dimension and outputs them to stdout so that you can pipe it into
# files. The elements are separated by '\t'.
#
# Synopsis:
# ./gen_matrices.py 4 4
#
# Author: Philipp Bohm

import sys
import random

try:
   dim_x, dim_y = int(sys.argv[1]), int(sys.argv[2])
except Exception as e:
   sys.stderr.write("error parsing matrix dimensions ...\n")
   raise

for row in range(0, dim_x):
   print("\t".join([str(random.uniform(0, 9999)) for x in range(0,
      ↪ dim_y)]))
```

Matrices were generated only once for each dimension.

3.3 Benchmark Script

Matrix creation, compilation, and execution were integrated into the `Test-Script.sh` automation pipeline.
Its responsibilities include:

1. Generating missing test matrices.

2. Compiling all four implementations: sequential, OpenMP, Pthreads, and MPI.
3. Executing each binary for each matrix size.
4. Redirecting numerical output to `/dev/null` to avoid I/O overhead during timing.
5. Capturing wall-clock runtime using POSIX `time`.

Only computational runtime was measured; printing intermediate matrices or debug information was disabled to eliminate I/O interference.

3.3.1 Key Modifications

The provided test utility was adapted as follows:

- The Python interpreter was changed from `python` to `python3`.
- Directory placeholders for matrix storage were initially empty; the `-f` flag was changed to `-s` to ensure that the file not only exists but is not empty.
- Matrix input files are only created when absent:
  – if a file does not exist, it is generated and stored in `data/`.
  – if a file already exists, it is reused.
- Benchmark matrices were generated for the following dimensions:

$$5 \times 4, \quad 10 \times 10, \quad 500 \times 500, \quad 2000 \times 2000, \quad 5000 \times 5000$$

Modified Script : Test-Script.sh

```
#!/bin/bash
echo "generate test-matrices with python if no test data found"
echo
# ----------- 5x4 and 4x5 -----------
if [ ! -f data/mat_5_4.txt ]; then
   echo "generate 5x4 matrix..."
   python3 random_float_matrix.py 5 4 > data/mat_5_4.txt
fi
if [ ! -f data/mat_4_5.txt ]; then
   python3 random_float_matrix.py 4 5 > data/mat_4_5.txt
fi

# ----------- 10x10 -----------
if [ ! -f data/mat_10x10a.txt ]; then
   echo "generate 10x10 matrix A..."
   python3 random_float_matrix.py 10 10 > data/mat_10x10a.txt
fi
if [ ! -f data/mat_10x10b.txt ]; then
   echo "generate 10x10 matrix B..."
```

```
   python3 random_float_matrix.py 10 10 > data/mat_10x10b.txt
fi

# ----------- 500x500 -----------
if [ ! -f data/mat_500x500a.txt ]; then
   echo "generate 500x500 matrix A..."
   python3 random_float_matrix.py 500 500 > data/mat_500x500a.txt
fi
if [ ! -f data/mat_500x500b.txt ]; then
   echo "generate 500x500 matrix B..."
   python3 random_float_matrix.py 500 500 > data/mat_500x500b.txt
fi

# ----------- 2000x2000 -----------
if [ ! -f data/mat_2000x2000a.txt ]; then
   echo "generate 2000x2000 matrix A..."
   python3 random_float_matrix.py 2000 2000 > data/mat_2000x2000a.
      ↪ txt
fi
if [ ! -f data/mat_2000x2000b.txt ]; then
   echo "generate 2000x2000 matrix B..."
   python3 random_float_matrix.py 2000 2000 > data/mat_2000x2000b.
      ↪ txt
fi

# ----------- 5000x5000 -----------
if [ ! -f data/mat_5000x5000a.txt ]; then
   echo "generate 5000x5000 matrix..."
   python3 random_float_matrix.py 5000 5000 > data/mat_5000x5000a.
      ↪ txt
fi
if [ ! -f data/mat_5000x5000b.txt ]; then
   python3 random_float_matrix.py 5000 5000 > data/mat_5000x5000b.
      ↪ txt
fi

echo "compile..."
echo
make
echo
echo "calculate..."
echo
echo "* * * * * * * 5x4 Matrix"
cal_t=$( { time bin/seq data/mat_5_4.txt data/mat_4_5.txt > /dev/
   ↪ null; } 2>&1 | grep real | awk '{print $2}' )
```

```
echo "with sequential $cal_t"

cal_t=$( { time bin/omp data/mat_5_4.txt data/mat_4_5.txt > /dev/
   ↪ null; } 2>&1 | grep real | awk '{print $2}' )
echo "with omp $cal_t"

cal_t=$( { time bin/thread2 data/mat_5_4.txt data/mat_4_5.txt > /dev
   ↪ /null; } 2>&1 | grep real | awk '{print $2}' )
echo "with thread2 $cal_t"

cal_t=$( { time mpirun -np 4 bin/mpi data/mat_5_4.txt data/mat_4_5.
   ↪ txt > /dev/null; } 2>&1 | grep real | awk '{print $2}' )
echo "with mpi(4p) $cal_t"
echo


echo "* * * * * * * 10x10 Matrix"
cal_t=$( { time bin/seq data/mat_10x10a.txt data/mat_10x10b.txt > /
   ↪ dev/null; } 2>&1 | grep real | awk '{print $2}' )
echo "with sequential $cal_t"

cal_t=$( { time bin/omp data/mat_10x10a.txt data/mat_10x10b.txt > /
   ↪ dev/null; } 2>&1 | grep real | awk '{print $2}' )
echo "with omp $cal_t"

cal_t=$( { time bin/thread2 data/mat_10x10a.txt data/mat_10x10b.txt
   ↪ > /dev/null; } 2>&1 | grep real | awk '{print $2}' )
echo "with thread2 $cal_t"

cal_t=$( { time mpirun -np 4 bin/mpi data/mat_10x10a.txt data/
   ↪ mat_10x10b.txt > /dev/null; } 2>&1 | grep real | awk '{print $2
   ↪ }' )
echo "with mpi(4p) $cal_t"
echo


echo "* * * * * * * 500x500 Matrix"
cal_t=$( { time bin/seq data/mat_500x500a.txt data/mat_500x500b.txt
   ↪ > /dev/null; } 2>&1 | grep real | awk '{print $2}' )
echo "with sequential $cal_t"

cal_t=$( { time bin/omp data/mat_500x500a.txt data/mat_500x500b.txt
   ↪ > /dev/null; } 2>&1 | grep real | awk '{print $2}' )
echo "with omp $cal_t"
```

```
cal_t=$( { time bin/thread2 data/mat_500x500a.txt data/mat_500x500b.
   ↪ txt > /dev/null; } 2>&1 | grep real | awk '{print $2}' )
echo "with thread2 $cal_t"

cal_t=$( { time mpirun -np 4 bin/mpi data/mat_500x500a.txt data/
   ↪ mat_500x500b.txt > /dev/null; } 2>&1 | grep real | awk '{print
   ↪ $2}' )
echo "with mpi(4p) $cal_t"
echo


echo "* * * * * * * 2000x2000 Matrix"
cal_t=$( { time bin/seq data/mat_2000x2000a.txt data/mat_2000x2000b.
   ↪ txt > /dev/null; } 2>&1 | grep real | awk '{print $2}' )
echo "with sequential $cal_t"

cal_t=$( { time bin/omp data/mat_2000x2000a.txt data/mat_2000x2000b.
   ↪ txt > /dev/null; } 2>&1 | grep real | awk '{print $2}' )
echo "with omp $cal_t"

cal_t=$( { time bin/thread2 data/mat_2000x2000a.txt data/
   ↪ mat_2000x2000b.txt > /dev/null; } 2>&1 | grep real | awk '{
   ↪ print $2}' )
echo "with thread2 $cal_t"

cal_t=$( { time mpirun -np 4 bin/mpi data/mat_2000x2000a.txt data/
   ↪ mat_2000x2000b.txt > /dev/null; } 2>&1 | grep real | awk '{
   ↪ print $2}' )
echo "with mpi(4p) $cal_t"
echo


echo "* * * * * * * 5000x5000 Matrix"
cal_t=$( { time bin/seq data/mat_5000x5000a.txt data/mat_5000x5000b.
   ↪ txt > /dev/null; } 2>&1 | grep real | awk '{print $2}' )
echo "with sequential $cal_t"

cal_t=$( { time bin/omp data/mat_5000x5000a.txt data/mat_5000x5000b.
   ↪ txt > /dev/null; } 2>&1 | grep real | awk '{print $2}' )
echo "with omp $cal_t"

cal_t=$( { time bin/thread2 data/mat_5000x5000a.txt data/
   ↪ mat_5000x5000b.txt > /dev/null; } 2>&1 | grep real | awk '{
   ↪ print $2}' )
echo "with thread2 $cal_t"
```

```
cal_t=$( { time mpirun -np 4 bin/mpi data/mat_5000x5000a.txt data/
   ↪ mat_5000x5000b.txt > /dev/null; } 2>&1 | grep real | awk '{
   ↪ print $2}' )
echo "with mpi(4p) $cal_t"
echo
```

In the terminal, following commands ensured the correct working of the script.

```
> nano Test-Script.sh # modify file: Test-Script.sh
> chmod +x Test-Script.sh # make executable
> ./Test-Script.sh # run the script
```

3.4 Results

Execution times for all implementations were collected using the `real` metric of the POSIX `time` utility.
Benchmark logs obtained from running the benchmark script using `./Test-Script.sh` are following:

```
islamabad@Namra:~/matrix_multiplication$ ./Test-Script.sh
generate test-matrices with python if no test data found

compile...

gcc -O2 -Wall -std=gnu99 -g -fopenmp -o bin/seq src/matrix.c src/sequential.c
gcc -O2 -Wall -std=gnu99 -g -fopenmp -o bin/omp src/matrix.c src/omp.c
gcc -O2 -Wall -std=gnu99 -g -fopenmp -pthread -o bin/thread2 src/matrix.c src/thread2.c
mpicc -O2 -Wall -std=gnu99 -g -fopenmp -o bin/mpi src/matrix.c src/mpi.c

calculate...

* * * * * * * 5x4 Matrix
with sequential    0m0.006s
with omp           0m0.014s
with thread2       0m0.007s
with mpi(4p)       0m0.081s

* * * * * * * 10x10 Matrix
with sequential    0m0.004s
with omp           0m0.007s
with thread2       0m0.004s
with mpi(4p)       0m0.079s

* * * * * * * 500x500 Matrix
with sequential    0m1.011s
with omp           0m0.785s
with thread2       0m0.758s
with mpi(4p)       0m0.071s

* * * * * * * 2000x2000 Matrix
with sequential    1m46.573s
with omp           0m54.399s
with thread2       0m57.713s
with mpi(4p)       0m0.094s

* * * * * * * 5000x5000 Matrix
with sequential    37m52.786s
with omp           22m48.886s
with thread2       19m49.032s
with mpi(4p)       0m0.247s
```

3.5 Performance Discussion

This scaling Matrix multiplication scales proportionally to n^3. As the matrix dimension grows, the number of arithmetic operations increases much faster than memory or communication costs. This characteristic determines which execution model becomes effective at different sizes.

Sequential Execution

1. **Small matrices (5×4, 10×10):** Sequential execution performs best because there is no setup cost and the total amount of work is minimal. The entire program runs within one core's cache and completes almost instantly.

2. **Medium matrices (500×500):** The number of arithmetic operations increases drastically: $500^3 = 125,000,000$ multiply–accumulate operations. The single-threaded implementation becomes slower because all computation is performed by one core. Parallel approaches begin to benefit from distributing work, though sequential execution still incurs no synchronization overhead.

3. **Large matrices (2000×2000, 5000×5000):** $2000^3 = 8 \times 10^9$ operations, $5000^3 = 125 \times 10^9$ operations. At this scale, one core cannot sustain the cubic workload; runtime grows sharply and sequential performance falls behind all parallel models. It remains useful as a correctness baseline, but not as a scalable solution.

OpenMP

1. **Small matrices:** For very small cases, OpenMP introduces thread creation and scheduling overhead without providing meaningful benefit. The workload per thread is too small to justify parallelization.
2. **Medium matrices:** Each thread receives enough contiguous rows to perform useful work. Because all threads access the same shared memory, communication overhead is minimal. OpenMP becomes faster than sequential and often provides consistent speedup
3. **Large matrices:** OpenMP continues to benefit from available cores, but the shared-memory design eventually limits performance. Threads compete for access to input matrices, and memory bandwidth becomes the main bottleneck. At this point, increasing thread count no longer improves throughput significantly.

Pthreads Pthreads provide explicit thread management. Unlike OpenMP, there is no abstraction layer: the programmer decides how rows are partitioned, when to synchronize, and where to allocate buffers. This brings both control and responsibility.

1. **Small matrices:** Thread creation costs exceed the time needed to multiply the matrices. Context-switch cost exceeds computation, making Pthreads slower than both sequential and OpenMP.
2. **Medium matrices:** When matrix blocks are assigned carefully, Pthreads avoid some of the scheduling overhead found in OpenMP. Each worker operates on its own portion of the matrix, and data reuse in cache becomes predictable. Performance becomes competitive and often exceeds OpenMP when threads stay active for long enough.
3. **Large matrices:** It benefits from explicit partitioning and stable worker lifetimes. Since the programmer controls synchronization and work distribution, the model avoids unnecessary runtime decisions. As a result, Pthreads can outperform OpenMP when the workload is heavy and well balanced.

MPI

1. **Small matrices:** MPI is inefficient. Splitting data between processes, sending it through MPI calls, and collecting results introduces more cost than the computation itself.
2. **Medium matrices:** As the workload per process increases, communication becomes a smaller part of the total runtime. Each process multiplies its own matrix block independently, and performance begins to exceed shared-memory approaches.

3. **Large matrices:** At sufficiently large workloads, MPI is most effective. Each rank computes locally with no shared-memory contention, and communication happens only at defined stages. Since computation dominates, distributed execution scales well.

Summary Parallelism is beneficial only when there is enough work to justify overhead. Small matrices run best sequentially. Medium matrices favor shared-memory approaches. Very large workloads benefit from distributed computation.

4 Merge Sorting

Sorting is a fundamental operation in computer science and numerical workloads. In contrast to matrix multiplication, which exhibits cubic computational complexity, classical merge sort achieves optimal asymptotic performance of $\mathcal{O}(n \log n)$ using a divide-and-conquer paradigm. The input array is recursively decomposed into halves until each segment consists of a single element; subsequently, segments are merged back in sorted order. This makes merge sort a suitable candidate for parallel decomposition over independent subranges, mirroring the workload partitioning principles explored in the matrix multiplication experiment.

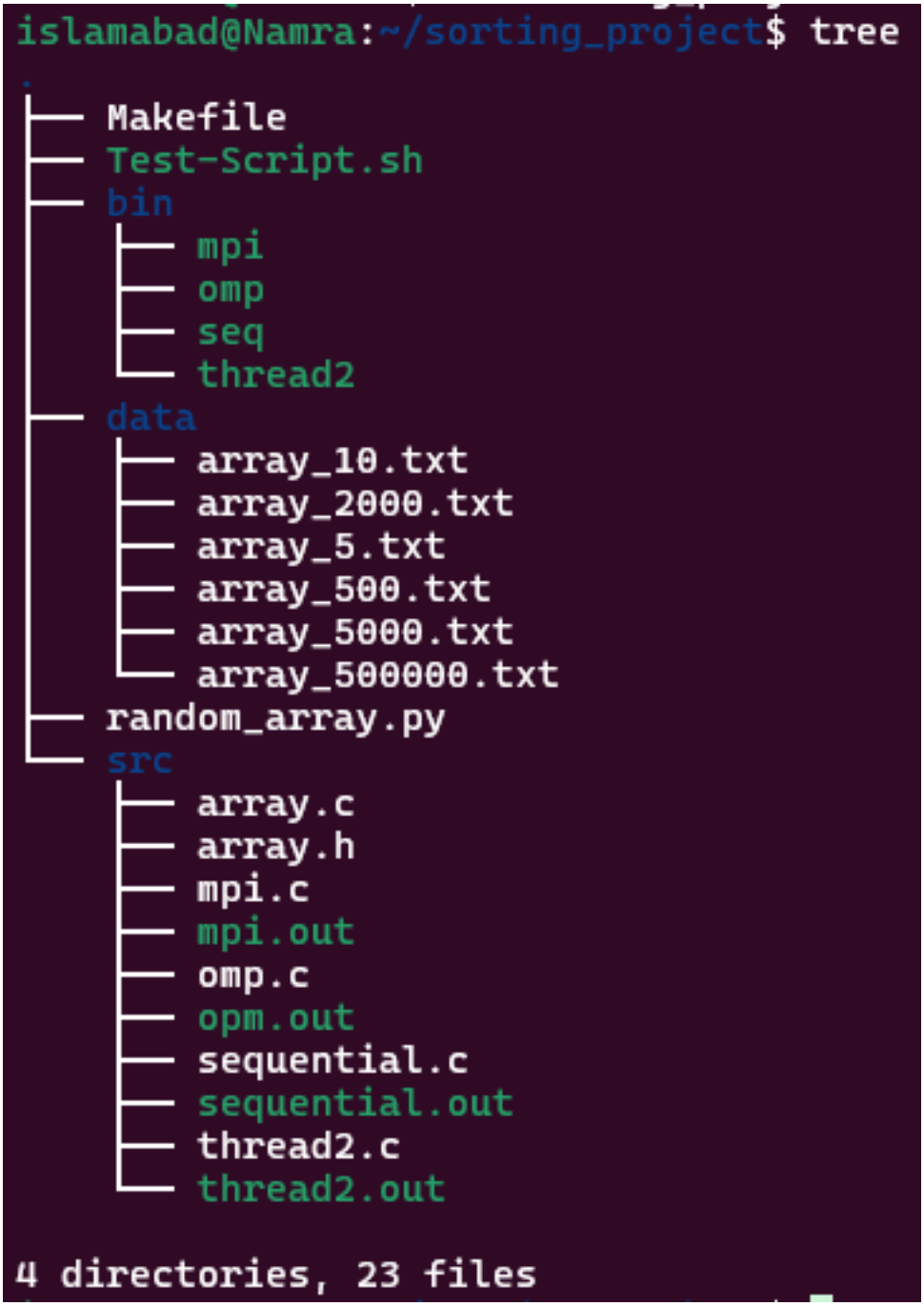
To investigate how different execution models affect performance, a standardized sorting project was created with explicit directory structure, build automation, and modular source organization. The project contains shared utilities for array acquisition and printing, as well as four execution models: sequential, POSIX threads, OpenMP, and MPI.

4.1 Project Layout

The experiment begins with a reproducible workspace that enforces separation between sources, data, and compiled binaries. The following directory as well as file structure is created:

- **src/** : All C source files including the sequential and parallel implementations.
- **bin/** : Contains compiled binary executables.
- **data/** : Contains test input arrays of different sizes.
- **random_array.py** : Python script to generate random integer arrays of arbitrary lengths. These test datasets are stored as tab-separated integers in the `data/` directory.
- **Makefile** : Defines compilation targets for all versions `bin/seq`, `bin/omp`, `bin/thread2`, `bin/mpi`
- **Test-Script.sh** : Automation script for input generation, compilation, execution, and benchmarking

** All the files are build on the same style as matrix multiplication script but for arrays sorting.*



4.2 Makefile - Shared Compilation Strategy

All implementations are compiled from a unified codebase to ensure performance comparability. A `Makefile` automates compilation across the four execution models using model-appropriate flags.

This standardized build pipeline removes compilation bias - every binary is generated with the strongest non-aggressive optimization flags. Platform dependencies are also abstracted, letting the benchmarking process focus purely on algorithmic differences.

Script: Makefile

```
CC = gcc
CFLAGS = -Wall -std=gnu99 -g -fopenmp
LIBS = src/array.c
TUNE = -O2

all: sequential omp thread2 mpi

sequential:
	$(CC) $(TUNE) $(CFLAGS) -o bin/seq $(LIBS) src/sequential.c
omp:
	$(CC) $(TUNE) $(CFLAGS) -o bin/omp $(LIBS) src/omp.c
thread2:
	$(CC) $(TUNE) $(CFLAGS) -pthread -o bin/thread2 $(LIBS) src/
	   ↪ thread2.c
mpi:
	mpicc $(TUNE) $(CFLAGS) -o bin/mpi $(LIBS) src/mpi.c
```

4.3 Array Acquisition and Shared Utilities

All models consume identical input arrays. To guarantee experiment reproducibility, a dedicated utility module ingests an unsorted integer dataset and encapsulates it within a uniform data structure. The procedure is:

1. Open the input file containing the integer list.
2. Count the total number of integers (one per line).
3. Allocate a contiguous buffer of size n and populate it.
4. Return an `array_struct` pointer with both data and size.

This structure ensures consistency in dataset ingestion and guarantees identical input across all execution models. The utility module also provides:

- A simple array pretty-printer.
- A destructor routine that frees both the integer buffer and the owning struct.

Script: array.c

```
#include <stdio.h>
#include <stdlib.h>
#include "array.h"

array_struct* get_array_struct(char filename[]) {
   FILE *f = fopen(filename, "r");
   if(!f) { printf("Error opening file.\n"); exit(EXIT_FAILURE); }
```

```
   array_struct *arr = malloc(sizeof(array_struct));
   arr->size = 0;
   int val;

   // Count lines
   while(fscanf(f, "%d", &val) == 1) arr->size++;
   rewind(f);

   arr->data = malloc(arr->size * sizeof(int));
   for(unsigned int i = 0; i < arr->size; i++) {
   if (fscanf(f, "%d", &arr->data[i]) != 1) {
      fprintf(stderr, "Error reading element %u from file.\n", i);
      exit(EXIT_FAILURE);
   }
}
   fclose(f);
   return arr;
}
void print_array(array_struct *arr) {
   for(unsigned int i=0; i<arr->size; i++)
      printf("%d\t", arr->data[i]);
   printf("\n");
}
void free_array(array_struct *arr) {
   free(arr->data);
   free(arr);
}
```

Script: array.h

```
typedef struct {
   unsigned int size;
   int *data;
} array_struct;

array_struct* get_array_struct(char filename[]);
void print_array(array_struct *arr);
void free_array(array_struct *arr);
```

4.4 Sequential Merge Sort Implementation

The sequential implementation adheres strictly to the classical merge sort algorithm. It is intentionally unoptimized beyond algorithmic correctness, establishing a baseline from which parallel improvements can be measured.

Merge sort decomposes into two core operations:

1. **merge**: combine two sorted partitions into a single ordered segment.
2. **merge_sort**: recursively divide the array into halves, sort each half, and merge results.

Merge Operation Two temporary buffers store the left and right partitions. Pointers advance monotonically through both buffers, selecting the smaller candidate at each step. Residual elements are appended to the destination array. All temporary storage is freed on exit, preventing memory leakage.

Recursive Sort The array subdivision proceeds until the subproblem shrinks to a base case (single element). Because every merge step produces a strictly larger sorted segment, correctness flows upward through the recursion tree until the full array is sorted.

The program flow is:

1. Load input via the shared array utilities.
2. Call merge sort on range $[0, n-1]$.
3. Output the sorted result.
4. Release all allocated memory.

Script: sequential.c

```
#include <stdio.h>
#include <stdlib.h>
#include "array.h"

// Merge two sorted sections
static void merge(int *arr, int l, int m, int r) {
   int n1 = m - l + 1;
   int n2 = r - m;

   int *L = malloc(n1 * sizeof(int));
   int *R = malloc(n2 * sizeof(int));
   if(!L || !R){
      fprintf(stderr, "malloc failed\n");
      exit(EXIT_FAILURE);
   }
   for(int i = 0; i < n1; i++) L[i] = arr[l + i];
   for(int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];

   int i = 0, j = 0, k = l;
   while(i < n1 && j < n2)
      arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
```

```
    while(i < n1) arr[k++] = L[i++];
    while(j < n2) arr[k++] = R[j++];

    free(L);
    free(R);
}
static void merge_sort(int *arr, int l, int r) {
    if(l >= r) return;

    int m = l + (r - l) / 2;
    merge_sort(arr, l, m);
    merge_sort(arr, m + 1, r);
    merge(arr, l, m, r);
}
int main(int argc, char **argv) {
    if(argc != 2) {
        printf("Provide one input file.\n");
        exit(EXIT_FAILURE);
    }
    array_struct *arr = get_array_struct(argv[1]);

    merge_sort(arr->data, 0, arr->size - 1);

    print_array(arr);
    free_array(arr);
    return 0;
}
```

4.5 POSIX Threaded Merge Sort

Where the sequential model trusts recursion to divide work, the pthread implementation partitions the data *up front*. This approach offers full control over concurrency: the array is partitioned into uniformly sized disjoint subranges, each handled by a worker thread. Although, excessive thread count causes false sharing when adjacent partitions reside on the same cache line.

Thread Sorting Phase Each worker thread receives:

- A pointer to the shared global array.
- A left index boundary.
- A right index boundary.

Threads execute a private sequential merge sort on their assigned slice, eliminating synchronization until local sorting is complete. Because slices do not overlap, no locking or atomics are required. Cache locality improves because each worker repeatedly accesses the same address interval, avoiding false sharing.

Sequential Merge Phase After all threads terminate, a bottom-up merging stage integrates the sorted intervals. In the first pass, subarrays of size k are merged pairwise into subarrays of size $2k$. Each subsequent round doubles the interval size until the entire array is fully merged. This merging is intentionally serial, as cross-thread tree merges would require complicated synchronization and result in diminishing returns.

Script: thread2.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include "array.h"

typedef struct {
   int *arr;
   int left;
   int right;
} thread_data;

static void merge(int *arr, int l, int m, int r){
   int n1 = m - l + 1;
   int n2 = r - m;

   int *L = malloc(n1 * sizeof(int));
   int *R = malloc(n2 * sizeof(int));

   for(int i=0; i<n1; i++) L[i] = arr[l+i];
   for(int i=0; i<n2; i++) R[i] = arr[m+1+i];

   int i=0,j=0,k=l;
   while(i<n1 && j<n2)
      arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];

   while(i<n1) arr[k++] = L[i++];
   while(j<n2) arr[k++] = R[j++];

   free(L); free(R);
}
```

```
static void merge_sort_seq(int *arr, int l, int r){
   if(l >= r) return;

   int m = l + (r-l)/2;
   merge_sort_seq(arr, l, m);
   merge_sort_seq(arr, m+1, r);
   merge(arr, l, m, r);
}

static void *thread_sort(void *arg){
   thread_data *d = (thread_data*)arg;
   merge_sort_seq(d->arr, d->left, d->right);
   return NULL;
}

int main(int argc, char **argv){
   if(argc != 2){
      printf("Provide one input file.\n");
      exit(EXIT_FAILURE);
   }

   array_struct *arr = get_array_struct(argv[1]);

   int cores = sysconf(_SC_NPROCESSORS_ONLN);
   pthread_t *threads = malloc(cores * sizeof(pthread_t));
   thread_data *td = malloc(cores * sizeof(thread_data));

   int chunk = arr->size / cores;
   int start = 0;

   for(int i=0; i<cores; i++){
      td[i].arr = arr->data;
      td[i].left = start;

      td[i].right = (i == cores-1) ?
         arr->size - 1 :
         start + chunk - 1;

      pthread_create(&threads[i], NULL, thread_sort, &td[i]);
      start += chunk;
   }

   for(int i=0; i<cores; i++) pthread_join(threads[i], NULL);

   /*** Merge all chunks sequentially ***/
```

```
    int step = chunk;
    while(step < arr->size){
        int left = 0;
        while(left + step < arr->size){
            int mid = left + step -1;
            int right = (left + 2*step -1 < arr->size) ?
                        (left + 2*step -1) :
                        (arr->size-1);
            merge(arr->data, left, mid, right);
            left += 2*step;
        }
        step *= 2;
    }

    print_array(arr);
    free(threads);
    free(td);
    free_array(arr);
    return 0;
}
```

4.6 OpenMP Recursive Merge Sort

OpenMP accelerates merge sort by parallelizing recursion rather than imposing static partitions. Each recursive split can be delegated to a new parallel task as long as a predefined depth permits spawning. A *parallel section* encompasses two child calls: one handling the left half, the other the right. This preserves algorithmic structure and avoids manual thread management.

To prevent unbounded task creation, the implementation derives a maximum recursion depth $\log_2(T)$, where T equals the number of available OpenMP threads.This ensures tasks do not exceed hardware parallelism. Past this depth, the algorithm reverts to standard sequential merge sort.

Key Characteristics

- All workers operate in shared memory.
- Subproblems are non-overlapping, avoiding data conflicts.
- Merges occur only after both child tasks complete, preserving global ordering.

Recursive OpenMP models perform best when task granularity aligns with L2/L3 capacity; otherwise tasks finish too quickly and scheduling overhead dominates.

Program logic:

1. Load input using shared utilities.

2. Query the OpenMP runtime for total available threads.
3. Compute depth bound $\log_2(T)$.
4. Launch a single recursive parallel entry point.

Script: omp.c

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
#include "array.h"

static void merge(int *arr,int l,int m,int r){
   int n1 = m-l+1;
   int n2 = r-m;

   int *L=malloc(n1*sizeof(int));
   int *R=malloc(n2*sizeof(int));

   for(int i=0;i<n1;i++) L[i]=arr[l+i];
   for(int i=0;i<n2;i++) R[i]=arr[m+1+i];

   int i=0,j=0,k=l;
   while(i<n1 && j<n2)
      arr[k++] = (L[i]<=R[j])?L[i++]:R[j++];

   while(i<n1) arr[k++]=L[i++];
   while(j<n2) arr[k++]=R[j++];

   free(L); free(R);
}

static void merge_sort_parallel(int *arr,int l,int r,int depth){
   if(l >= r) return;

   int m = l + (r - l) / 2;
   if(depth > 0) {
      #pragma omp parallel sections
      {
         #pragma omp section
         merge_sort_parallel(arr,l,m,depth-1);

         #pragma omp section
         merge_sort_parallel(arr,m+1,r,depth-1);
      }
   } else {
```

```
      merge_sort_parallel(arr,l,m,0);
      merge_sort_parallel(arr,m+1,r,0);
   }
   merge(arr,l,m,r);
}

int main(int argc,char **argv){
   if(argc!=2){
      printf("Provide one input file.\n");
      exit(EXIT_FAILURE);
   }
   array_struct *arr = get_array_struct(argv[1]);

   int threads = omp_get_max_threads();
   /** log2(threads) safe parallel depth **/
   int depth = 0;
   while((1<<depth) < threads) depth++;

   #pragma omp parallel
   {
      #pragma omp single nowait
      merge_sort_parallel(arr->data,0,arr->size-1,depth);
   }

   print_array(arr);
   free_array(arr);
   return 0;
}
```

4.7 MPI Distributed Merge Sort

The MPI model distributes segments of the input array across independent processes, eliminating shared-memory contention entirely. The workflow is as follows:

Data Distribution Rank 0 (master) reads the full array, broadcasts the global size, and scatters equal partitions to workers. Each rank sorts its subarray locally using the sequential merge sort implementation.

Aggregation Once local segments are sorted, all results are gathered back to the master. The master performs iterative bottom-up merging across increasing block sizes until the full array is globally sorted. Since processes operate in isolated memory spaces, no synchronization is required during local sorting.

The primary limitation is communication overhead. Scattering, gathering and multi-stage

merging introduce network latency that dominates for small array sizes. Conversely, for larger datasets communication becomes negligible relative to computation, making MPI ideal for large workloads.

MPI is optimal only when the cost of scattering and gathering $\mathcal{O}(n)$ is dwarfed by local sorting $\mathcal{O}(n \log n)$, typically at large n.

Script: mpi.c

```
#include <stdio.h>
#include <stdlib.h>
#include <mpi.h>
#include "array.h"

static void merge(int *arr,int l,int m,int r){
   int n1 = m-l+1;
   int n2 = r-m;
   int *L = malloc(n1*sizeof(int));
   int *R = malloc(n2*sizeof(int));
   for(int i=0;i<n1;i++) L[i]=arr[l+i];
   for(int i=0;i<n2;i++) R[i]=arr[m+1+i];
   int i=0,j=0,k=l;
   while(i<n1 && j<n2)
      arr[k++] = (L[i]<=R[j])?L[i++]:R[j++];
   while(i<n1) arr[k++] = L[i++];
   while(j<n2) arr[k++] = R[j++];
   free(L); free(R);
}

static void merge_sort_seq(int *arr,int l,int r){
   if(l >= r) return;
   int m = l + (r-l)/2;
   merge_sort_seq(arr,l,m);
   merge_sort_seq(arr,m+1,r);
   merge(arr,l,m,r);
}

int main(int argc,char **argv){
   MPI_Init(&argc,&argv);
   int rank,size;
   MPI_Comm_rank(MPI_COMM_WORLD,&rank);
   MPI_Comm_size(MPI_COMM_WORLD,&size);

   int *arr=NULL;
   int n=0;
```

```
if(rank==0){
   if(argc!=2){
      printf("Provide one input file.\n");
      MPI_Abort(MPI_COMM_WORLD,1);
   }
   array_struct *a = get_array_struct(argv[1]);
   arr = a->data;
   n = a->size;
}

MPI_Bcast(&n,1,MPI_INT,0,MPI_COMM_WORLD);

int chunk = (n + size - 1) / size;

int *local = malloc(chunk*sizeof(int));

MPI_Scatter(arr,chunk,MPI_INT,local,chunk,MPI_INT,0,
   ↪ MPI_COMM_WORLD);

merge_sort_seq(local,0,chunk-1);

int *sorted=NULL;
if(rank==0) sorted = malloc(chunk*size*sizeof(int));

MPI_Gather(local,chunk,MPI_INT,sorted,chunk,MPI_INT,0,
   ↪ MPI_COMM_WORLD);

if(rank==0){
   int real=n;
   while(chunk < real){
      int left = 0;
      while(left + chunk < real){
         int mid = left + chunk -1;
         int right =
            (left + 2*chunk -1 < real)?
            (left + 2*chunk -1):
            (real-1);
         merge(sorted,left,mid,right);
         left += 2*chunk;
      }
      chunk*=2;
   }
   for(int i=0;i<n;i++) printf("%d\t",sorted[i]);
   printf("\n");
   free(sorted);
```

```
        free(arr);
    }

    free(local);
    MPI_Finalize();
    return 0;
}
```

4.8 Random Dataset Generator

The Python generator creates uniformly random integers across a fixed interval (1–10000). Output values are delimited with tabs to maintain compatibility with C input scanners. This approach guarantees reproducible datasets and allows controlled scaling of input size with a single parameter.

Script:random_array.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-

import sys
import random

if len(sys.argv) != 2:
    print("Usage: ./random_array.py <size>")
    sys.exit(1)

try:
    size = int(sys.argv[1])
except ValueError:
    print("Size must be an integer")
    sys.exit(1)

# generate random integers between 1 and 10000
arr = [str(random.randint(1, 10000)) for _ in range(size)]

# print to stdout separated by tabs
print("\t".join(arr))
```

4.9 Automated Test Script

A dedicated bash script automates dataset generation, compilation, and timed execution. The script ensures that each input size is consistently used across all implementations. Measured runtimes are printed without array output to avoid I/O artifacts.

The experimental workflow:

1. Generate random arrays of multiple sizes if not already present.
2. Invoke `make` to compile all binaries.
3. Time each implementation for multiple input sizes, including very small (5–10), medium (500–5000), and large (500000) arrays.

Script: Test-Script.sh

```
#!/bin/bash
echo "generate test-arrays with python if no test data found"
echo

if [ ! -f data/array_5.txt ]; then
   echo "generate array of size 5..."
   python3 random_array.py 5 > data/array_5.txt
fi

if [ ! -f data/array_10.txt ]; then
   echo "generate array of size 10..."
   python3 random_array.py 10 > data/array_10.txt
fi

if [ ! -f data/array_500.txt ]; then
   echo "generate array of size 500..."
   python3 random_array.py 500 > data/array_500.txt
fi

if [ ! -f data/array_2000.txt ]; then
   echo "generate array of size 2000..."
   python3 random_array.py 2000 > data/array_2000.txt
fi

if [ ! -f data/array_5000.txt ]; then
   echo "generate array of size 5000..."
   python3 random_array.py 5000 > data/array_5000.txt
fi

if [ ! -f data/array_500000.txt ]; then
   echo "generate array of size 500000..."
   python3 random_array.py 500000 > data/array_500000.txt
fi

echo "compile..."
make
```

```
echo
echo "calculate..."
echo

echo "* * * * * * * Array size 5"
cal_t=$((time bin/seq data/array_5.txt) 2>&1 > /dev/null | grep real
    ↪  | awk '{print $2}')
echo "with sequential $cal_t"

cal_t=$((time bin/omp data/array_5.txt) 2>&1 > /dev/null | grep real
    ↪  | awk '{print $2}')
echo "with omp $cal_t"

cal_t=$((time bin/thread2 data/array_5.txt) 2>&1 > /dev/null | grep
    ↪ real | awk '{print $2}')
echo "with thread2 $cal_t"

cal_t=$((time mpirun -np 4 bin/mpi data/array_5.txt) 2>&1 > /dev/
    ↪ null | grep real | awk '{print $2}')
echo "with mpi(4p) $cal_t"
echo

echo "* * * * * * * Array size 10"
cal_t=$((time bin/seq data/array_10.txt) 2>&1 > /dev/null | grep
    ↪ real | awk '{print $2}')
echo "with sequential $cal_t"

cal_t=$((time bin/omp data/array_10.txt) 2>&1 > /dev/null | grep
    ↪ real | awk '{print $2}')
echo "with omp $cal_t"

cal_t=$((time bin/thread2 data/array_10.txt) 2>&1 > /dev/null | grep
    ↪  real | awk '{print $2}')
echo "with thread2 $cal_t"

cal_t=$((time mpirun -np 4 bin/mpi data/array_10.txt) 2>&1 > /dev/
    ↪ null | grep real | awk '{print $2}')
echo "with mpi(4p) $cal_t"
echo

echo "* * * * * * * Array size 500"
cal_t=$((time bin/seq data/array_500.txt) 2>&1 > /dev/null | grep
    ↪ real | awk '{print $2}')
echo "with sequential $cal_t"
```

```
cal_t=$((time bin/omp data/array_500.txt) 2>&1 > /dev/null | grep
   ↪ real | awk '{print $2}')
echo "with omp $cal_t"

cal_t=$((time bin/thread2 data/array_500.txt) 2>&1 > /dev/null |
   ↪ grep real | awk '{print $2}')
echo "with thread2 $cal_t"

cal_t=$((time mpirun -np 4 bin/mpi data/array_500.txt) 2>&1 > /dev/
   ↪ null | grep real | awk '{print $2}')
echo "with mpi(4p) $cal_t"
echo

echo "* * * * * * * Array size 2000"
cal_t=$((time bin/seq data/array_2000.txt) 2>&1 > /dev/null | grep
   ↪ real | awk '{print $2}')
echo "with sequential $cal_t"

cal_t=$((time bin/omp data/array_2000.txt) 2>&1 > /dev/null | grep
   ↪ real | awk '{print $2}')
echo "with omp $cal_t"

cal_t=$((time bin/thread2 data/array_2000.txt) 2>&1 > /dev/null |
   ↪ grep real | awk '{print $2}')
echo "with thread2 $cal_t"

cal_t=$((time mpirun -np 4 bin/mpi data/array_2000.txt) 2>&1 > /dev/
   ↪ null | grep real | awk '{print $2}')
echo "with mpi(4p) $cal_t"
echo

echo "* * * * * * * Array size 5000"
cal_t=$((time bin/seq data/array_5000.txt) 2>&1 > /dev/null | grep
   ↪ real | awk '{print $2}')
echo "with sequential $cal_t"

cal_t=$((time bin/omp data/array_5000.txt) 2>&1 > /dev/null | grep
   ↪ real | awk '{print $2}')
echo "with omp $cal_t"

cal_t=$((time bin/thread2 data/array_5000.txt) 2>&1 > /dev/null |
   ↪ grep real | awk '{print $2}')
echo "with thread2 $cal_t"

cal_t=$((time mpirun -np 4 bin/mpi data/array_5000.txt) 2>&1 > /dev/
```

```
   ↪ null | grep real | awk '{print $2}')
echo "with mpi(4p) $cal_t"
echo

echo "* * * * * * * Array size 500000"
cal_t=$((time bin/seq data/array_500000.txt) 2>&1 > /dev/null | grep
   ↪  real | awk '{print $2}')
echo "with sequential $cal_t"

cal_t=$((time bin/omp data/array_500000.txt) 2>&1 > /dev/null | grep
   ↪  real | awk '{print $2}')
echo "with omp $cal_t"

cal_t=$((time bin/thread2 data/array_500000.txt) 2>&1 > /dev/null |
   ↪ grep real | awk '{print $2}')
echo "with thread2 $cal_t"

cal_t=$((time mpirun -np 4 bin/mpi data/array_500000.txt) 2>&1 > /
   ↪ dev/null | grep real | awk '{print $2}')
echo "with mpi(4p) $cal_t"
echo
```

4.10 Results

Execution times for all implementations were collected using the `real` metric of the POSIX `time` utility.
Benchmark logs obtained from running the benchmark script using `./Test-Script.sh` are following:

```
islamabad@Namra:~/sorting_project$ ./Test-Script.sh
generate test-arrays with python if no test data found

compile...
gcc -O2 -Wall -std=gnu99 -g -fopenmp -o bin/seq src/array.c src/sequential.c
gcc -O2 -Wall -std=gnu99 -g -fopenmp -o bin/omp src/array.c src/omp.c
gcc -O2 -Wall -std=gnu99 -g -fopenmp -pthread -o bin/thread2 src/array.c src/thread2.c
mpicc -O2 -Wall -std=gnu99 -g -fopenmp -o bin/mpi src/array.c src/mpi.c

calculate...

* * * * * * * Array size 5
with sequential    0m0.004s
with omp           0m0.006s
with thread2       0m0.005s
with mpi(4p)       0m0.190s

* * * * * * * Array size 10
with sequential    0m0.004s
with omp           0m0.015s
with thread2       0m0.005s
with mpi(4p)       0m0.070s

* * * * * * * Array size 500
with sequential    0m0.004s
with omp           0m0.006s
with thread2       0m0.004s
with mpi(4p)       0m0.071s

* * * * * * * Array size 2000
with sequential    0m0.007s
with omp           0m0.014s
with thread2       0m0.008s
with mpi(4p)       0m0.077s

* * * * * * * Array size 5000
with sequential    0m0.010s
with omp           0m0.011s
with thread2       0m0.008s
with mpi(4p)       0m0.094s

* * * * * * * Array size 500000
with sequential    0m0.327s
with omp           0m0.246s
with thread2       0m0.176s
with mpi(4p)       0m0.061s
```

4.11 Performance Discussion

Merge sort maintains a theoretical runtime of $O(n \log n)$, which grows more moderately than matrix multiplication. However, practical performance still depends on memory access patterns, recursion depth, and the cost of coordinating parallel workers.

Sequential Execution The sequential model acts as the control baseline. It performs no thread creation, no synchronization, and no inter-process communication; every CPU cycle directly contributes to sorting and merging.

1. **Small arrays (5, 10, 500):** Sequential merge sort completes faster than parallel models because no thread or process management occurs. Each recursive split is shallow, and

merges require little work.

2. **Medium arrays (2000, 5000):** The recursion depth grows and merge work accumulates, but the single-threaded implementation remains competitive. There are no scheduling barriers or context switches. Although parallel models begin to provide advantages, but their overhead can still mask gains.

3. **Large arrays (500000):** Only at very large inputs does the sequential model reaches its limits. The fixed $O(n \log n)$ curve eventually dominates, and a single-core cannot sustain the sorting workload. Parallel approaches finally begin to outperform sequential execution.

OpenMP The OpenMP implementation parallelizes the recursive structure itself. When the array splits into left and right halves, the runtime may spawn parallel tasks, allowing work to flow naturally down the recursion tree. This provides a high-level abstraction, but thread orchestration remains internally managed by the OpenMP runtime.

1. **Small arrays:** The recursion tree is shallow and splits trivial subranges. Task creation, thread pool activation, and runtime overhead outweigh the sorting work. OpenMP is slower than sequential execution at this scale.

2. **Medium arrays:** When recursion generates enough independent subproblems, OpenMP can keep multiple cores busy. Performance improves, though merging remains partially serial and reduces ideal speedup.

3. **Large arrays:** At large scales, OpenMP becomes effective. Subtasks become deep and heavy, allowing the runtime to maintain useful concurrency. However, shared-memory contention and global merges eventually impose bottlenecks. OpenMP improves substantially over sequential, but is outperformed by manually partitioned models that avoid the overhead of runtime scheduling.

Pthreads The pthread-based implementation conducts parallelism through explicit slice partitioning. Each worker thread sorts a contiguous interval using a private merge sort. Only after workers terminate is a bottom-up merge performed on the global array. This approach trades elegance for control, but rewards predictable cache locality.

1. **Small arrays:** Creating many worker threads is wasted effort. Each thread sorts a very small slice, so lifecycle cost dominates. The final merge phase, though trivial, cannot recover the wasted startup time and performance falls behind sequential execution.

2. **Medium arrays:** Pthreads gain advantage when each worker receives a sizable partition. Sorting contiguous blocks improves locality, and thread management overhead becomes minimal relative to useful work. Pthreads often match or exceed OpenMP in this range.

3. **Large arrays:** Long-lived worker threads process large slices with minimal runtime overhead, and data remains contiguous throughout. The final merge step still requires

time, but the local sort duration makes threaded decomposition worthwhile. The explicit control offered by Pthreads produces consistent performance.

MPI MPI distributes array segments across isolated processes. Each rank performs a full sequential merge sort on its private slice, eliminating shared-memory contention entirely. Only the master process performs global merging using iterative gather operations.

1. **Small arrays:** MPI is not suitable for small inputs. Sending data between processes costs more than sorting it locally. All ranks spend more time communicating than sorting.
2. **Medium arrays:** Even for moderate sizes, process communication may outweigh computation. The benefits of distributed sorting appear only when each process receives enough data.
3. **Large arrays:** With very large inputs, local sorting time far exceeds communication overhead. Each rank processes its own slice independently, and merging occurs only after sorting. In this case, MPI scales well and can outperform shared-memory parallelism.

Summary The experimental behavior mirrors theoretical expectations:

- **Small arrays:** Parallel models are penalized by overhead; Sequential is ideal.
- **Medium arrays:** Thread-based models offer improvement when partitions are large enough.
- **Large arrays:** Distributed approaches provide the strongest performance because communication is amortized across heavy computation.

5 Conclusion

The results clearly show that parallelism is only beneficial when the amount of useful work outweighs the cost of coordinating workers. For very small inputs, all parallel approaches are overhead-bound, and sequential execution remains fastest because no threads or processes need to be created, synchronized, or communicated with. As problem size increases, shared-memory models become effective: OpenMP offers easy concurrency and reliable mid-range performance, while Pthreads benefits from explicit partitioning and consistently outperforms OpenMP once workloads are stable and large enough to amortize thread creation.

Distributed execution via MPI shows the strongest scaling but only at large sizes. Communication overhead dominates at the small end, but once computation becomes heavy, private memory and process-level independence avoid cache contention entirely, allowing MPI to surpass all other models. Merge sort (memory-bound) and matrix multiplication (compute-bound) reinforce the same principle: performance is determined not only by parallelism, but by how well the algorithm's structure matches the execution model. There is no universally "best" approach—choosing the right model is a matter of aligning workload characteristics with the architecture that exploits them most efficiently.

Code Repository

The full source code for matrix multiplication and merge sort experiments is available at:

- **Modified Project Repository:**

  Namra-5/parallel_sorting_and_matrix_benchmark: Comparative implementation of sequential, OpenMP, POSIX threads, and MPI models to perform merge sort and matrix multiplication. Includes automated benchmarking scripts, input generators, and performance analysis. (n.d.). https://github.com/Namra-5/parallel_sorting_and_matrix_benchmark

- **Original Matrix Multiplication Repository:**

  mperlet/matrix_multiplication: Parallel Matrix Multiplication Using OpenMP, Phtreads, and MPI. (n.d.). https://github.com/mperlet/matrix_multiplication